

```
//User accepting the request for permission.
auth.getSession().startAuthentication(MainActivity.this);

//Return to your app
@Override
protected void onResume() {
    super.onResume();
    if(auth.getSession().authenticationSuccessful()) {
        try {
            auth.getSession().finishAuthentication();
            AccessTokenPair tok=auth.getSession().
getAccessTokenPair();
            storeKeys(tok.key, tok.secret); //store the user's
tokens somewhere for reuse.
        }catch(Exception e){Log.d("info", "error", e);}
    }
}
```



Note: In the last step, you override the `onResume` function, which is the default callback when the control returns to your app after authentication.

At this point, you should have the Dropbox API set up on your Android system, and your session should also be authenticated. If you have the user's tokens stored somewhere, it is recommended that you start off this section by reinitialising the access keys, to avoid user reauthentication. You can do this by using the following lines of code:

```
AccessTokenPair access = getStoredKeys();
//retrieve the stored tokens.
auth.getSession().setAccessTokenPair(access);
```



Note: In order to keep things simple, this tutorial has not implemented the `storeKeys` and `getStoredKeys` methods. They should ideally be implemented when developing actual apps, as explained above.

Let's now look at the most important part of the API—the file operations using Dropbox.

Uploading a file

You can simulate reading a file using the `ByteArrayInputStream` to operate on the contents of a given string. Any file on your device may also be used, by specifying the path to the file. A call to the `putFile` method uploads the file to Dropbox. The code for uploading a file is shown below:

```
String content="Hello World!";
ByteArrayInputStream ip=new
ByteArrayInputStream(content.getBytes());
Entry e1=auth.putFile("/test.txt", ip, content.
length(), null, null);
```

Downloading a file

Downloading is simple—use the `getFile` method to download your file, and store it on your Android device. The code to download a file from Dropbox (the same one you just uploaded) is:

```
File f=new File(getFilesDir()+"/test.txt");
if(!f.exists())
    f.createNewFile();
is=new FileOutputStream(f);
DropboxFileInfo e1=auth.getFile("/test.txt", null, is,
null);
```

Obtaining file metadata

This is arguably the most useful feature of Dropbox. Most applications work by making decisions based on the *metadata* of a given file. The Dropbox API provides a method called *metadata*, which returns information related to the file, like *last modified*, *revision*, *path*, etc. This information is handy when you want to use previous versions of the file, or when you need some form of version control. A sample use of the *metadata* method is as follows:

```
Entry meta = auth.metadata("/test.txt", 1, null, false,
null);
```

Now, putting everything together, the complete code for the `MainActivity.java` of your Android project is available at: http://www.linuxforu.com/article_source_code/march13/Dropbox_Android.zip.

Writing apps that use the Dropbox API is straightforward. The ease of use and flexibility of the API make it a great way to provide cloud support for your app. The API has support for many different platforms like Android, iOS, Python and Ruby, and also has many third-party APIs, which extend Dropbox support for most other platforms as well. When you face a problem using the API, a great resource to look at is the Dropbox Developers' site, at <https://www.dropbox.com/developers/start/>, which has a detailed overview about the use of the API.

Finally, the Dropbox API for Android is a quick and powerful way of allowing your applications to take advantage of cloud storage. Android, being a prominent mobile development platform, needs tools to provide seamless integration with cloud services, and the Dropbox API does exactly this. 

By: Sahil Chelaramani

The author is an open source activist, who loves Linux, Python and Android. He is a third-year student at the Goa Engineering College (GEC).